

DATA MINING AND MACHINE LEARNING

LIVRABLE : RAPPORT LE PROJET DE DESSALEMENT

Réalisé par :
Khmaich oualid



Table des matières

Introduction	3
Problématique.....	3
Chapitre 1 – Traitement et nettoyage des données	5
Nettoyage avancé et correction des anomalies capteurs	10
Chapitre 2 : Modélisation prédictive – Approches Machine Learning.....	12
Définition du problème et des variables	12
Modèles linéaires de référence	12
Modèles non linéaires avancés	12
Analyse comparative et choix du modèle	13
Analyse	14
Chapitre 3 : Stratégie Backwash – Analyse et Modélisation	15
Objectif	15
Préparation des données	15
Sélection et optimisation des modèles	15
Création des indices physiques	16
Décision backwash	16
Sélection des filtres à nettoyer.....	17
Visualisation et vérification.....	17
Analyse	18
Chapitre 4 : Préparation du dataset pour mieux optimiser la décision du Backwash :	21
Étapes principales :	21
Évaluation des performances.....	22
Chapitre 5 : Pipeline de prédiction Machine learning en cascade	27
Organisation générale	27
Fonction extract_features_from_df.....	27
Fonction predict_tank_and_prod	27
Chapitre 6 : le Pipeline avec Deep Learning.....	29
1. Préparation des données	29
2. Définition du modèle DL :	29
3. Optimisation des hyperparamètres avec GridSearchCV	29
4. Visualisation des performances	30
5. Entraînement final	30
6. Évaluation et sauvegarde	30
Le pipeline final :	32

Pipeline du premier modèle (Tank) :.....	32
Création du second modèle (Production finale) :	32
Pipeline complet en cascade :	33
Chapitre 7 : Optimisation entre Machine learning et Deep learning.....	34
Recherche de la combinaison optimale	34
la fonction generate_all_combinations	35
Résultats : Machine learning.....	36
Résultats : Deep learning	39
Conclusion :	41

Introduction

La pénurie d'eau est aujourd'hui un problème majeur à l'échelle mondiale. Au Maroc, cette situation devient de plus en plus critique à cause du changement climatique, de la baisse des précipitations et de l'augmentation des besoins en eau. Dans ce contexte, le dessalement de l'eau de mer représente une solution essentielle pour assurer un approvisionnement continu en eau potable et industrielle.

Les usines de dessalement reposent sur des procédés techniques complexes, principalement basés sur la filtration par membranes. Ces membranes fonctionnent souvent sous des conditions de flux élevés afin de maximiser la production. Toutefois, ce mode de fonctionnement favorise l'apparition de l'encrassement des membranes, appelé **fouling**. Ce phénomène réduit l'efficacité du système, augmente la consommation énergétique et diminue la durée de vie des filtres.

Pour limiter les effets du fouling, les usines utilisent des cycles de nettoyage réguliers, dont le **Backwash**. Ce procédé consiste à arrêter temporairement la production afin de nettoyer les membranes et restaurer leurs performances. Néanmoins, la gestion du backwash pose un problème important : un nettoyage trop fréquent entraîne une perte de production, tandis qu'un nettoyage insuffisant accélère l'encrassement et dégrade les équipements.

Avec l'évolution des systèmes industriels, les usines de dessalement génèrent aujourd'hui de grandes quantités de données à partir de capteurs. Ces données peuvent être exploitées grâce aux techniques de **Machine Learning** et de **Deep Learning** pour mieux comprendre le comportement du système et prévoir l'évolution du fouling. Ces méthodes permettent ainsi d'optimiser le cycle de backwash en trouvant un équilibre entre arrêt de production, consommation énergétique et rendement final.

Ce projet a pour objectif d'appliquer des approches basées sur l'intelligence artificielle à un dataset réel de grande taille afin d'améliorer la gestion du processus de dessalement. L'objectif final est de proposer une solution plus efficace, durable et adaptée aux contraintes réelles des usines de dessalement.

Problématique

Dans une usine de dessalement, la décision de déclencher un cycle de **Backwash** est un choix critique. Cette décision influence directement la production d'eau, la consommation énergétique et la durée de vie des membranes. En pratique, le backwash est souvent déclenché selon des règles fixes ou des seuils prédéfinis, sans prendre pleinement en compte l'état réel du système ni l'évolution simultanée de l'ensemble des paramètres capteurs.

Cette approche statique devient insuffisante face à la complexité du procédé et à la grande variabilité des conditions de fonctionnement. Le défi principal est donc de **déterminer, à partir**

des données capteurs, le moment optimal pour lancer le backwash, de manière à éviter un encrassement excessif tout en maximisant la production finale.

La problématique de ce travail consiste ainsi à répondre à la question suivante :

comment exploiter les données industrielles massives à l'aide du Machine Learning et du Deep Learning pour optimiser la décision de backwash et atteindre un équilibre optimal entre performance de production et préservation des membranes ?

Objectifs du travail

L'objectif général de ce projet est d'exploiter les techniques de **Data Science**, de **Machine Learning** et de **Deep Learning** afin d'analyser le fonctionnement réel d'une usine de dessalement et d'optimiser la décision de **Backwash** à partir des données capteurs.

Plus précisément, ce travail vise à :

- **Analyser et comprendre les données industrielles** issues des capteurs (91 variables, 500 000 lignes) afin d'identifier les paramètres les plus influents sur l'encrassement des membranes et la performance du système.
- **Décortiquer le comportement du procédé** à travers des analyses statistiques et exploratoires (corrélations, tendances, anomalies), afin de mieux comprendre les relations entre flux, pression, production et fouling.
- **Construire des modèles de Machine Learning** capables de prédire l'évolution de l'état des membranes et l'impact d'un backwash sur la production et la consommation énergétique.
- **Utiliser des modèles de Deep Learning** pour capturer les comportements non linéaires et les dynamiques temporelles du système, difficiles à modéliser par des méthodes classiques.
- **Aider à la prise de décision opérationnelle**, en proposant une stratégie de backwash basée sur des prédictions plutôt que sur des seuils fixes, permettant d'atteindre un meilleur compromis entre continuité de production et préservation des équipements.

À travers ces objectifs, ce travail cherche à démontrer l'apport concret de l'intelligence artificielle dans l'optimisation des procédés industriels complexes, en particulier dans le domaine du dessalement de l'eau.

Chapitre 1 – Traitement et nettoyage des données

1.1 Compréhension du procédé industriel

Avant toute analyse, une compréhension du procédé de dessalement a été réalisée afin de donner un sens physique aux données capteurs. Le processus étudié suit une chaîne industrielle classique :

- **Prise d'eau (Water Intake)** : aspiration de l'eau de mer contenant sable, sel et impuretés.
- **Préfiltration (HMMF)** : élimination des particules grossières afin de protéger les équipements en aval.
- **Stockage** : l'eau filtrée est stockée dans des réservoirs avant pompage.
- **Pompage haute pression** : mise sous forte pression nécessaire au fonctionnement de l'osmose inverse.
- **Trains RO** : huit trains identiques composés de filtres cartouches et de membranes RO.
- **Séparation finale** : production de perméat (eau douce) et rejet de la saumure.

Cette compréhension a permis d'organiser les données selon la logique réelle du procédé.

1.2 Description du dataset

Le dataset utilisé est issu d'un environnement industriel réel de dessalement par osmose inverse.

- **Taille** : 572 645 lignes × 92 colonnes (~402 MB)
- **Types de données** :
 - 91 variables numériques (float)
 - 1 variable temporelle (time)
- **Valeurs manquantes** : aucune
- **Données multitrains** : trains A à H (systèmes identiques)

Les variables couvrent l'ensemble du processus : préfiltration, pompage, membranes RO, perméat et rejet.

1.3 Interprétation des capteurs et structuration sémantique

Un travail de décodage des noms de colonnes a été réalisé à partir des **tags industriels** :

- **FIT** : débit (Flow)

- **PIT** : pression
- **PDIT** : différence de pression
- **AIT** : paramètres analytiques (pH, conductivité)
- **LIT** : niveau des réservoirs

Les lettres A–H correspondent aux différents trains de production. Cette étape a permis de relier chaque variable à un point précis du procédé physique.

1.4 Prétraitement temporel et intégrité des données

Les actions suivantes ont été appliquées :

- Conversion de la colonne time au format **datetime**
- Définition du temps comme **index principal**
- Vérification des **doublons** (aucun doublon détecté)
- Vérification de la cohérence des types et de la structure globale du dataset

Ces étapes garantissent la fiabilité de l'analyse temporelle et des modèles prédictifs.

1.5 Réorganisation des données selon le processus

Les colonnes ont été réorganisées pour suivre l'ordre logique du procédé industriel, en deux grandes parties :

1. **Prétraitement et stockage**
 - Intake
 - Filtres HMMF (débits, pressions, ΔP)
 - Réservoirs et backwash HMMF
2. **Osmose inverse et production finale**
 - Filtres cartouches
 - Pompes RO
 - Membranes RO (pressions feed / reject)
 - Débit de perméat

Cette structuration permet une lecture claire du cheminement de l'eau et facilite l'analyse causale entre les variables.

1.6 Premières analyses exploratoires

Enfin, des analyses exploratoires initiales ont été menées afin de préparer la phase de modélisation :

- Statistiques descriptives (min, max, moyenne, dispersion)
- Visualisation des débits et pressions
- Analyse temporelle du perméat
- Étude des corrélations entre variables critiques liées au fouling

Ces analyses constituent la base de compréhension nécessaire pour la construction des modèles de Machine Learning et Deep Learning.

Gestion des doublons temporels et agrégation des mesures

Lors de l'analyse du dataset, il a été constaté que certaines observations partageaient le même instant temporel à la **minute près**, bien que leurs valeurs capteurs diffèrent. Ce phénomène est fréquent dans les systèmes industriels où plusieurs acquisitions peuvent être réalisées au cours d'une même minute.

Une analyse spécifique a donc été menée afin d'évaluer la cohérence de ces mesures redondantes. Les données ont d'abord été regroupées par minute, puis, pour chaque capteur, les valeurs **minimales, maximales et moyennes** ont été calculées. La différence absolue entre les valeurs extrêmes a ensuite été comparée à un seuil de **10 % de la valeur moyenne**, conformément aux recommandations pédagogiques, afin d'identifier d'éventuelles incohérences significatives.

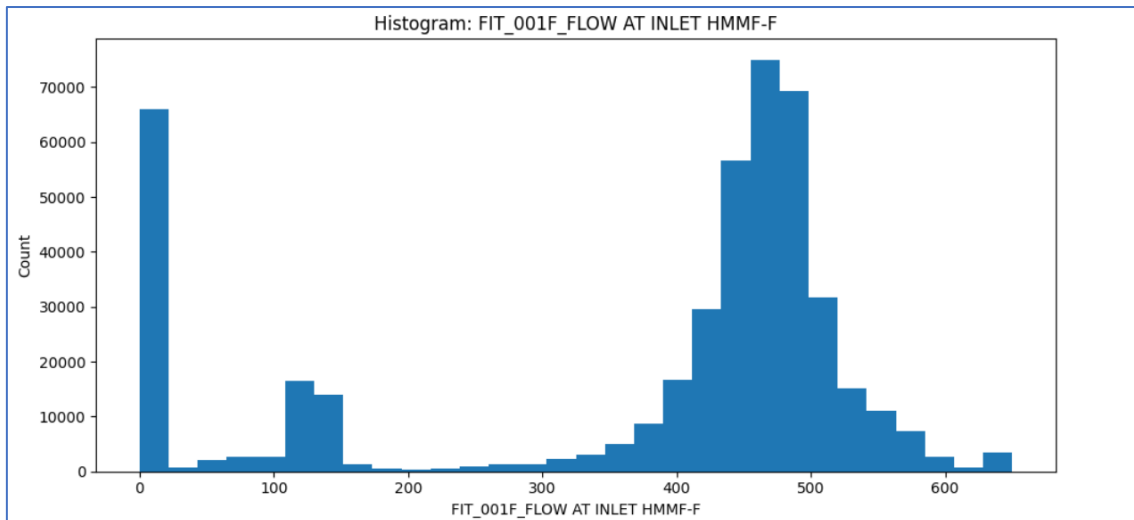
Cette analyse a permis :

- d'identifier les capteurs présentant des variations notables au sein d'une même minute,
- de quantifier l'amplitude relative des écarts,
- de vérifier que ces variations restaient globalement compatibles avec la dynamique normale du procédé.

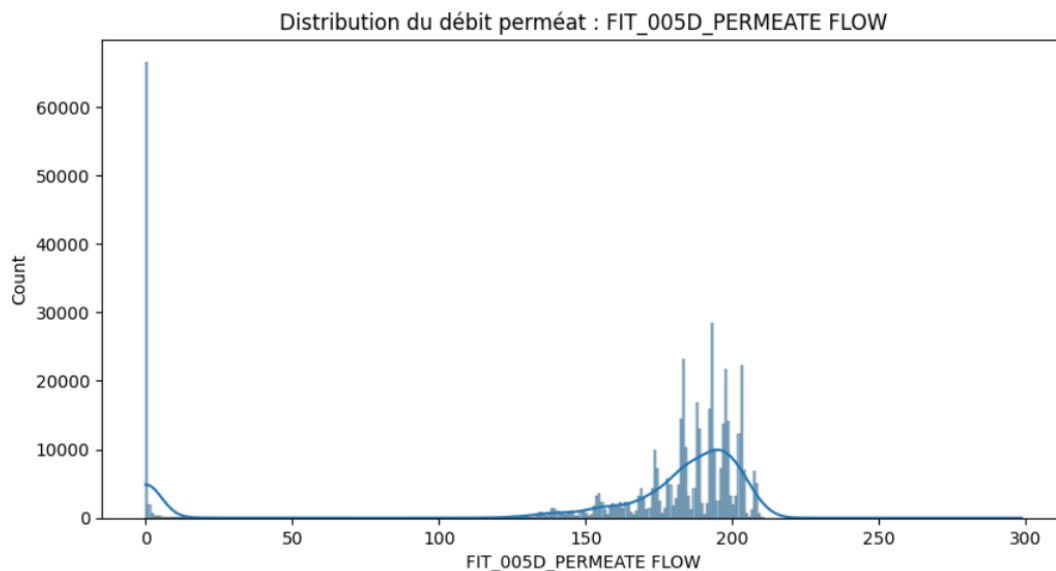
Afin d'unifier la résolution temporelle du dataset et de préparer les données à la modélisation, une **agrégation par minute** a été appliquée. Pour chaque minute, la moyenne des variables numériques a été retenue comme valeur représentative. Cette approche permet de :

- supprimer les redondances temporelles,
- réduire le bruit lié aux micro-variations,
- conserver une information fidèle au comportement global du système.

Le dataset final obtenu est ainsi homogène sur le plan temporel, cohérent sur le plan physique et directement exploitable pour les phases d'analyse exploratoire avancée et de modélisation en Machine Learning et Deep Learning.



puis après on s'est trouvé avec le problème des zéros non détectés erronés par les capteurs



◆ Analyse du graphique : FIT_005F_PERMEATE FLOW

Axe X : valeurs du débit (0 à 350) Axe Y : nombre d'occurrences Observation : Pic massif à 0 (presque 60 000 occurrences) Zone "en cloche" entre 150–250

Interprétation : Pic à 0 Très fréquent → représente des périodes sans production. Causes possibles : Capteurs indiquant 0 lors de la mise en veille, Arrêts manuels du système (maintenance, test, changement d'étape), Pannes temporaires ou coupures, Ou encore des tests opératoires où le système tourne à vide. ➡ Donc, ce pic traduit un état OFF ou non productif.

■ Cloche (150–250) Forme régulière, centrée autour de ~200 → fonctionnement stable. Représente la phase active de production d'eau dessalée. ➡ C'est l'état ON du système.

Conclusion Partie Signification Interprétation Pic à 0 État OFF Arrêt, test, panne ou capteur inactif Cloche 150–250 État ON Fonctionnement stable du système

➡ La distribution est bimodale : Une “mode” à 0 (arrêt / tests / pannes) Une autre vers 200 (production effective)

Détection automatique des différents cas (arrêts, tests, pannes...)

On peut appliquer un traitement de **clustering** pour distinguer les différents comportements du système à partir des données mesurées.

💡 Pourquoi sur les mesures ?

Deux approches possibles :

1 Analyse simple (rapide)

- Tu appliques le clustering **seulement sur la variable du débit**.
 - Cela permet de repérer :
 - Le **cluster autour de 0** → arrêts, tests, ou capteur inactif.
 - Le **cluster autour de 200** → fonctionnement normal.
- 👉 Idéal pour une première analyse rapide.

2 Analyse avancée (multi-capteurs)

- Tu appliques le clustering sur **plusieurs mesures** (ex. débit, pression, pH, conductivité...).
- Cela permet de mieux distinguer :
 - Un **arrêt normal ou planifié**,
 - Une **panne ou un dysfonctionnement**,
 - Une **phase de test manuelle**.

⚙️ En résumé

Objectif	Type d'analyse	Variables utilisées	Résultat
Rapide	Simple	1 variable (débit)	Détection ON/OFF
Précise	Avancée	Plusieurs capteurs	Détection de pannes, tests, arrêts

Nettoyage avancé et correction des anomalies capteurs

Après l'agrégation temporelle, une phase de nettoyage avancé a été appliquée afin de garantir la cohérence physique des mesures et de limiter l'impact des anomalies capteurs sur la modélisation.

Correction basée sur des seuils physiques

Des **plages de fonctionnement réalistes** ont été définies pour les principales familles de capteurs (débits, pressions, pH, conductivité, niveaux). Chaque variable a été automatiquement associée à une famille selon son nom, puis les valeurs situées hors de ces plages ont été remplacées par une valeur par défaut physiquement acceptable. Cette étape permet d'éliminer les valeurs impossibles dues à des erreurs de mesure ou de transmission.

Traitement des zéros suspects

Une analyse spécifique a été menée sur les valeurs nulles. Certains zéros ne correspondent pas à un arrêt réel du procédé mais à des **défaillances capteurs ponctuelles**. Ces zéros sont détectés lorsqu'une variable est proche de zéro alors que la majorité des autres variables de la même ligne présentent des valeurs élevées.

Dans ce cas, la valeur a été remplacée par la **médiane de la colonne**, choix robuste adapté aux distributions asymétriques observées dans les données industrielles.

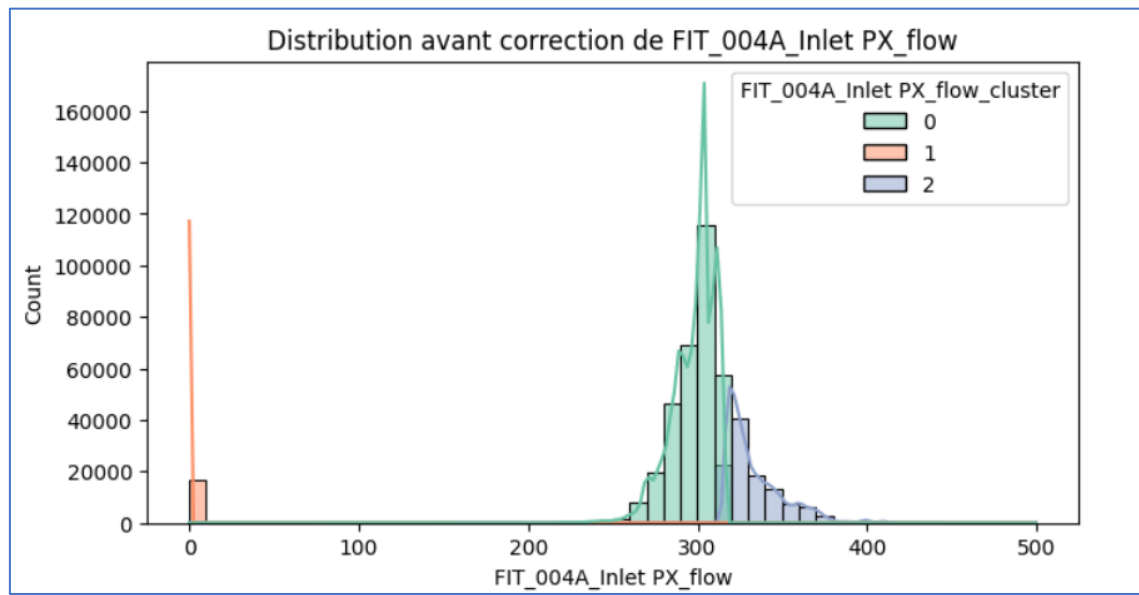
Validation par clustering

Afin de valider la cohérence des corrections, une approche non supervisée par **K-means (k = 3)** a été utilisée sur les distributions des variables. Les clusters ont permis de distinguer les régimes normaux de fonctionnement des groupes anormaux dominés par des valeurs nulles ou aberrantes. Les corrections ont été vérifiées visuellement à l'aide d'histogrammes avant et après traitement, confirmant une meilleure séparation des régimes physiques réalistes.

Dataset final

À l'issue de cette phase, le dataset obtenu est :

- nettoyé des valeurs hors domaine physique,
- corrigé des zéros non représentatifs,
- cohérent sur le plan industriel,
- directement exploitable pour l'analyse exploratoire avancée et la modélisation en Machine Learning et Deep Learning.



Chapitre 2 : Modélisation prédictive – Approches Machine Learning

Définition du problème et des variables

Dans une première phase de modélisation, l'objectif a été de **prédire le niveau du réservoir d'eau filtrée (LIT_002_FILTER_WATER_STG_TNK)**, variable clé reflétant l'état amont du procédé et influençant directement la continuité de la production.

Les variables explicatives ont été sélectionnées à partir de la **première partie du processus** (prétraitement, HMMF, réservoir), en excluant la variable cible afin d'éviter toute fuite d'information. Le dataset a ensuite été séparé en ensembles d'entraînement et de test selon un ratio **80 % / 20 %**.

Modèles linéaires de référence

Trois modèles linéaires ont été testés afin d'établir une base de comparaison :

- Régression linéaire
- Ridge Regression
- Lasso Regression

Les résultats obtenus montrent des performances limitées, avec un **R² maximal de l'ordre de 0,35**. Ces scores indiquent que les relations entre les variables capteurs et le niveau du réservoir sont **fortement non linéaires**, ce qui dépasse les capacités explicatives des modèles linéaires classiques.

Modèles non linéaires avancés

Afin de mieux capturer la complexité du procédé industriel, des modèles basés sur les **arbres de décision et l'ensemble learning** ont été évalués :

- Random Forest Regressor
- Gradient Boosting Regressor
- XGBoost Regressor

Une recherche par **Grid Search avec validation croisée** a été appliquée pour optimiser les hyperparamètres.

Les résultats montrent une nette amélioration des performances, en particulier pour le **Random Forest**, qui atteint :

- **R² ≈ 0,96**
- **RMSE ≈ 2,09**
- **MAE ≈ 1,17**

Ces performances confirment la capacité du modèle à reproduire fidèlement le comportement réel du système à partir des données capteurs.

Analyse comparative et choix du modèle

Modèle	Meilleurs paramètres	R ²	MSE	RMSE	MAE
Random Forest	max_depth=None, n_estimators=100	0.9597	4.3678	2.0899	1.1657
Gradient Boosting	learning_rate=0.1, max_depth=5, n_estimators=100	0.6751	35.1729	5.9307	4.3681
XGBoost	learning_rate=0.1, max_depth=5, n_estimators=100	0.6772	34.9515	5.9120	4.3568

La comparaison entre modèles linéaires et non linéaires met en évidence que :

- les modèles linéaires sont insuffisants pour représenter le procédé,
- les modèles d'ensemble capturent efficacement les interactions complexes entre variables,
- le **Random Forest** offre le meilleur compromis entre précision, robustesse et interprétabilité.

Ce modèle est ainsi retenu comme **référence** pour la suite du travail, notamment pour l'analyse de l'impact des variables amont sur la dynamique du système et la préparation à l'optimisation du backwash.

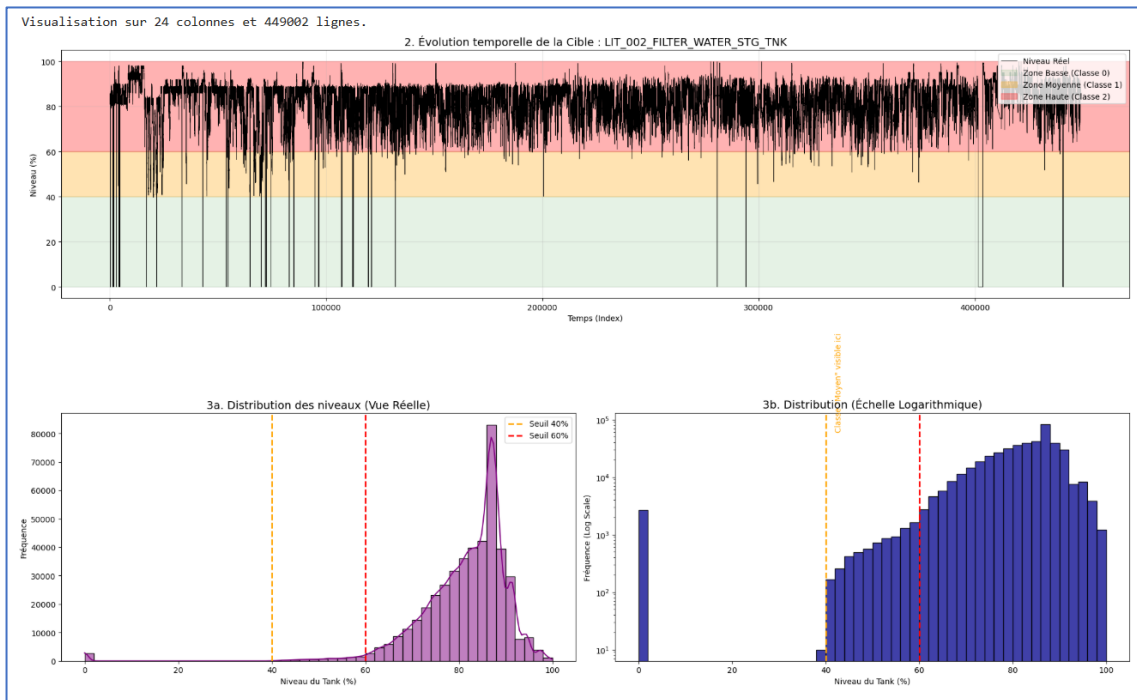
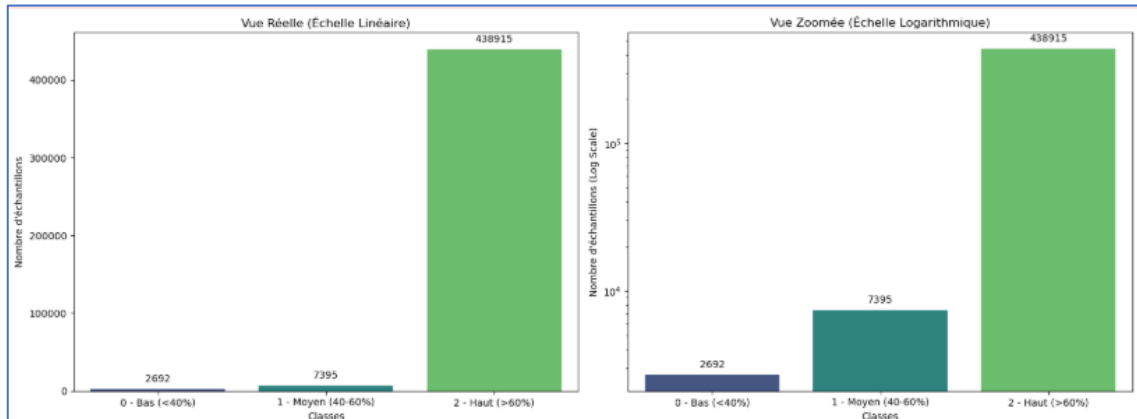
Classification par KNN sur les niveaux du réservoir

- **Objectif** : Discrétiser le niveau du réservoir LIT_002_FILTER_WATER_STG_TNK en trois classes : Bas (<40%), Moyen (40–60%), Haut (>60%).
- **Données** : 449 002 échantillons, 23 features.
- **Distribution des classes** :
 - Bas : 2 692
 - Moyen : 7 395
 - Haut : 438 915
- **Méthode** : K-Nearest Neighbors (k=5), standardisation appliquée aux features, train/test split 80/20.
- **Résultats sur le test set** :
 - Accuracy globale : 99,56%
 - Rapport détaillé :
 - Bas : précision 0,99, rappel 0,98, F1-score 0,98

- Moyen : précision 0,89, rappel 0,85, F1-score 0,87
- Haut : précision 1,00, rappel 1,00, F1-score 1,00

Analyse :

Le modèle prédit très bien la classe majoritaire (Haut), mais les classes minoritaires (Bas et Moyen) sont moins précises et rappellent moins bien, conséquence directe du **déséquilibre très marqué des classes**.



Chapitre 3 : Stratégie Backwash – Analyse et Modélisation

Objectif

L'objectif est de créer une **stratégie intelligente de backwash** pour le système HMMF :

- Décider **quand déclencher un backwash** selon le niveau du réservoir et l'état des filtres.
- Déterminer **quels filtres nettoyer** dans le cas d'un backwash partiel.
- Optimiser le procédé pour **maximiser la production** tout en évitant un encrassement critique des filtres.

Préparation des données

- **Dataset** : data_ready_to_use.csv
- **Target** : LIT_002_FILTER_WATER_STG_TNK (niveau du réservoir)
- **Features** : toutes les colonnes avant la cible dans le dataset

Scaling : Toutes les features sont standardisées (StandardScaler) pour :

- Les modèles KNN et SVM qui sont sensibles aux échelles différentes.
- Optimisation de la performance et convergence plus rapide.

Sélection et optimisation des modèles

Modèles testés pour prédire le niveau du réservoir

Modèle	Paramètres testés	R ² sur échantillon
KNN	n_neighbors=[3,5,9], weights=['uniform','distance']	0.6644
SVM	C=[1,10,100], epsilon=[0.1,0.2]	0.5826

- **Échantillon** : 15 000 lignes pour limiter la consommation RAM et accélérer GridSearchCV.
- **Résultat** : **KNN gagnant**, meilleur R² 0,6644, paramètres optimaux : n_neighbors=5, weights='distance'.

Entraînement final

- **Dataset complet** : 449 002 lignes
- **Split Train/Test** : 80% / 20%
- **Performance finale KNN** :
 - R^2 : 0,9532
 - MAE : 1,2410
 - RMSE : 2,2514

Conclusion : KNN prédit très précisément le niveau du réservoir, ce qui est critique pour déclencher la stratégie de backwash.

Création des indices physiques

Pour décider le backwash, on crée trois indices basés sur les colonnes PDIT et FIT :

Indice	Description	Calcul
INDICE_ENCRASSEMENT_HMMF	Encrassement global des filtres	Moyenne des ΔP de tous les filtres PDIT_001A...J
INDICE_IMPACT_PRODUCTION	Impact sur le débit	INDICE_ENCRASSEMENT_HMMF / DEBIT_ENTREE_HMMF (DEBIT_ENTREE_HMMF = moyenne des débits d'entrée FIT_001A...J)
INDICE_DESEQUILIBRE_FILTERES	Déséquilibre entre filtres	Écart-type des ΔP des filtres (montre quels filtres sont plus encrassés)

Ces indices permettent de décider si un backwash est nécessaire, partiel ou complet.

On va les intégrer dans la prediction de la production finale, et puis on va essayer de trouver les meilleures combinaisons d'eux qui nous donne la meilleure production finale et sans trop de dégâts :

Décision backwash

Décision encodée numériquement :

Code	Décision
0	PAS_DE_BACKWASH
1	RETARDER_BACKWASH
2	BACKWASH_PARTIEL

Code	Décision
3	BACKWASH_COMPLET

Règles appliquées :

1. **Niveau réservoir < 40%** → priorité production :
 - Pas de backwash (code 0)
2. **Niveau 40–60%** → backwash partiel possible :
 - Si capacité de stockage faible → retarder backwash (1)
 - Si déséquilibre des filtres élevé → backwash partiel (2)
 - Sinon → pas de backwash (0)
3. **Niveau ≥ 60%** → backwash complet possible :
 - Si capacité stockage faible → retarder (1)
 - Si encrassement global élevé → backwash complet (3)
 - Sinon → pas de backwash (0)

Raisonnement :

- Éviter de backwasher quand le réservoir est trop bas (risque production).
- Prioriser le nettoyage partiel quand certains filtres sont plus sales.
- Autoriser backwash complet uniquement si le réservoir est suffisant et filtres critiques.

Sélection des filtres à nettoyer

Pour **backwash partiel (code 2)** :

- On choisit les **filtres les plus encrassés**, définis comme le **top 30 %** des ΔP (np.percentile).

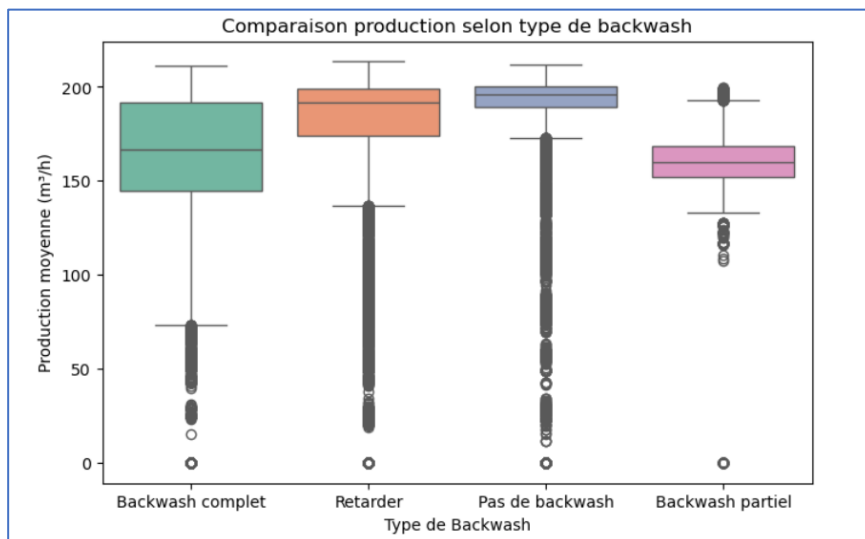
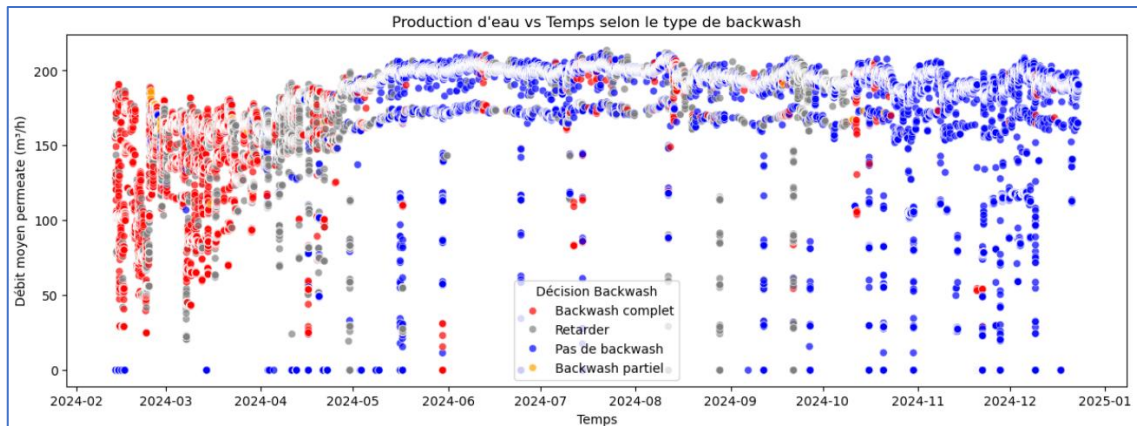
Pour **backwash complet (code 3)** :

- Tous les filtres sont nettoyés.

Fonction `filtres_a_backwasser_numeric` retourne une liste des filtres concernés selon le type de backwash.

Visualisation et vérification

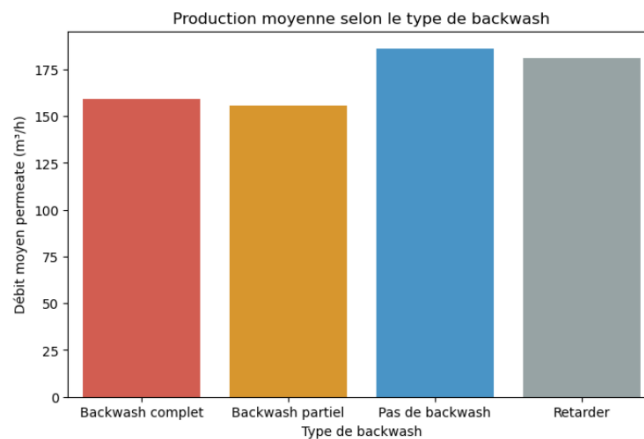
- Création d'indices pour chaque ligne.
- Attribution d'un code de décision pour chaque ligne.
- Identification des filtres à nettoyer pour les backwash partiels et complets.



Analyse_:

Cette analyse porte sur les performances de production d'eau (perméat) en fonction des cycles de nettoyage (**backwash**) dans votre installation de dessalement.

Difficulté de décider : (avec le trade-off **d'endommagement si on retarde et on ne fait pas**)



1. Analyse Temporelle (Scatter Plot)

Le premier graphique montre l'évolution du débit moyen de perméat sur l'année 2024.

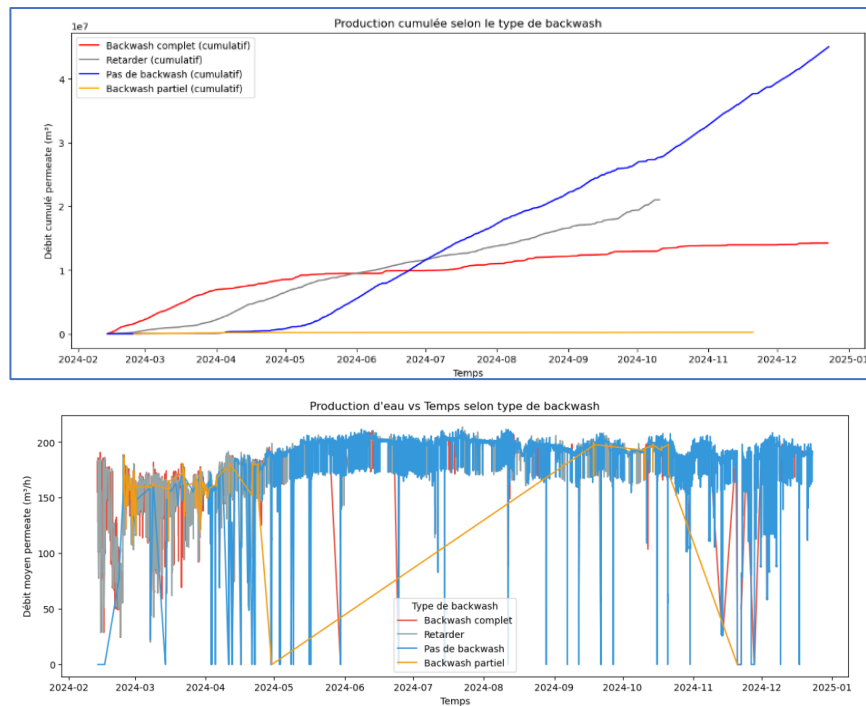
- **Phase de montée en charge (Février - Avril) :** On observe une instabilité initiale avec des débits fluctuant . Les points rouges (**Backwash complet**) sont fréquents, suggérant une phase de mise en route ou un colmatage rapide dû à une eau brute chargée.
- **Stabilisation (Mai - Décembre) :** La production se stabilise dans une plage haute . La majorité des points sont bleus (**Pas de backwash**), ce qui indique un régime de croisière efficace.
- **Comportement des Backwashes :** Les décisions de backwash (complet ou partiel) coïncident souvent avec des chutes de débit ou précèdent des remontées de performance.

2. Analyse Statistique (Boxplot)

Le boxplot permet de comparer l'efficacité opérationnelle selon la décision prise.

- **Pas de backwash (Bleu) :** C'est ici que la production est la plus élevée et la plus stable (médiane proche de 195 C'est l'état nominal de l'installation.)
- **Retarder (Orange) :** La médiane reste haute presque 190, ce qui suggère que la stratégie de "retarder" le lavage est souvent appliquée lorsque les performances sont encore acceptables, sans dégradation immédiate critique.
- **Backwash complet (Vert) & Partiel (Rose) :** Ces deux catégories affichent des médianes nettement plus basses
 - **Interprétation :** Le backwash est déclenché *parce que* le débit a chuté (encrassement des membranes). Le grand nombre d'outliers (points vers le bas) sous ces boîtes confirme que ces décisions sont prises en réaction à une baisse de performance significative.

L'analyse visuelle des débits de perméat **ne permet pas** de quantifier l'efficacité réelle des stratégies de backwash, car elle illustre une réponse opérationnelle plutôt qu'une causalité directe. Les graphiques ignorent le rendement net de l'installation, omettant le volume d'eau produit qui est réutilisé pour les phases de nettoyage des membranes. De plus, l'impact de l'indisponibilité technique liée à la durée de chaque cycle de lavage n'est pas intégré dans la simple observation des débits instantanés. La préservation de l'intégrité des membranes à long terme constitue une variable critique non détectable visuellement, mais essentielle pour la viabilité économique du procédé. En conclusion, **une étude comparative rigoureuse nécessite le calcul d'indicateurs de performance (KPIs), tels que le Volume Net Produit, le niveau d'encastrement des filtres,..., pour départager scientifiquement l'efficacité de ces décisions**



Chapitre 4 : Préparation du dataset pour mieux optimiser la décision du Backwash :

Objectif : Structurer et enrichir les données pour analyser les cycles de backwash et leur impact sur la production et les filtres HMMF.

Étapes principales :

1. **Production** : agrégation des débits FIT005A–H en Production_totale.
2. **Pression différentielle** : calcul de la moyenne des filtres DP_moyen_filtres.
3. **Variables Backwash** : ajout de type_backwash, duree_backwash, intervalle_min, freq_backwash, Niveau_reservoir.
4. **Identification des cycles** : regroupement des backwash consécutifs et calcul de leur durée.
5. **Intervalles et fréquence** : calcul des intervalles entre backwash et de la fréquence quotidienne.
6. **Dataset final** : sélection des colonnes essentielles (production, indices, ΔP filtres) et sauvegarde pour analyse.

Résultat : dataset consolidé, cohérent et prêt pour l'analyse exploratoire et la modélisation prédictive.

La colonne **FILTRES_A_BACKWASHER** a été convertie en liste Python afin de manipuler correctement les filtres concernés par chaque backwash. Tous les filtres uniques présents dans le dataset ont été identifiés et des colonnes binaires ont été créées pour chacun, indiquant si le filtre a été nettoyé lors du backwash. La variable **Production_totale** a été renommée en **target** et repositionnée à la fin du dataset afin de faciliter son utilisation comme variable cible pour les analyses et modèles prédictifs. Le dataset final contient désormais les colonnes indicatrices de filtres ainsi que la variable cible prête pour la modélisation.

	F-F	SURE ACROSS HMMF-G	SURE ACROSS HMMF-H	SURE ACROSS HMMF-I	SURE ACROSS HMMF-J	HMMF-A	HMMF-B	HMMF-C
1	147	0.6707500219345093	0.5246250033378601	0.6661249995231628	0.7645000219345093	1	1	1
2	112	0.6697499752044678	0.5239999890327454	0.6657500267028809	0.763999985694885	1	1	1
3	135	0.668749988079071	0.5218750238418579	0.6642500162124634	0.7628750205039978	1	1	1
4	129	0.6690000295639038	0.5224999785423279	0.6631249785423279	0.761500009536743	1	1	1
5	177	0.6691250205039978	0.5214999914169312	0.6641250252723694	0.762374997138977	1	1	1
6	159	0.6194999814033508	0.4729999899864197	0.6206250190734863	0.7127500176429749	1	1	1
7	139	0.637624979019165	0.4952499866485595	0.6324999928474426	0.7278749942779541	1	1	1
8	166	0.6679999828338623	0.5211250185966492	0.6631249785423279	0.7618749737739563	1	1	1
9	146	0.6694999933242798	0.5236250162124634	0.6652500033378601	0.762499988079071	1	1	1

	time	Niveau_reservoir	SION_BACKWASH_NUM	DP_moyen_filtres	type_backwash	duree_backwash	intervalle_min	freq_backw
1	2024-02-13 11:38:00	82.89375305175781	3	0.6954750120639801	3	271.0		73
2	2024-02-13 11:39:00	82.375	3	0.6944249987602233	3	271.0	1.0	73
3	2024-02-13 11:40:00	81.35624694824219	3	0.6937000036239624	3	271.0	1.0	73
4	2024-02-13 11:41:00	81.48750305175781	3	0.6924499988555908	3	271.0	1.0	73
5	2024-02-13 11:42:00	81.23750305175781	3	0.6930875062942505	3	271.0	1.0	73
6	2024-02-13 11:43:00	81.9749984741211	3	0.6198375076055527	3	271.0	1.0	73
7	2024-02-13 11:44:00	81.9749984741211	3	0.6596500009298325	3	271.0	1.0	73
8	2024-02-13 11:45:00	84.1187515258789	3	0.6867499947547913	3	271.0	1.0	73
9	2024-02-13 11:46:00	83.8375015258789	3	0.6886400041340029	3	271.0	1.0	73

La phase de modélisation prédictive a été structurée pour maximiser la précision dans la prédiction de la production totale d’eau.

- **Chargement et préparation des données** : Les données sont triées chronologiquement, avec séparation des variables explicatives et de la cible target pour un apprentissage supervisé fiable.
- **Nettoyage et normalisation** : Les valeurs manquantes sont imputées par la moyenne et toutes les variables sont standardisées pour homogénéiser les échelles, améliorant la convergence des modèles.
- **Sélection et optimisation des modèles** : Trois approches sont testées :
 - Régression **Ridge** pour une base linéaire régularisée
 - **K-Nearest Neighbors** pour capturer les relations locales
 - **HistGradientBoosting** pour modéliser les interactions complexes non linéairesChaque modèle est optimisé via **GridSearchCV** avec validation croisée temporelle (TimeSeriesSplit), permettant de rechercher automatiquement les hyperparamètres optimaux tout en respectant l’ordre temporel des données et en évitant le **data leakage**.

```
--- Entraînement : Ridge ---
--- Entraînement : KNN ---
--- Entraînement : HistGB ---

--- SYNTHÈSE DES PERFORMANCES ---
Modèle      R2      RMSE      MAE
0 Ridge  0.545548  265.321530  234.248073
1 KNN    0.186262  355.034811  161.780186
2 HistGB 0.917395  113.118020  71.567263
4/6 - Génération des courbes d'optimisation...
```

Évaluation des performances : Les modèles sont évalués sur le jeu de test selon le R^2 , RMSE et MAE. Les courbes générées pour Ridge, KNN et

HistGradientBoosting illustrent l'impact des hyperparamètres sur la performance, offrant une visualisation claire de l'optimisation.

En effet, les trois modèles montrent des comportements très différents sur ce dataset.

Ridge :

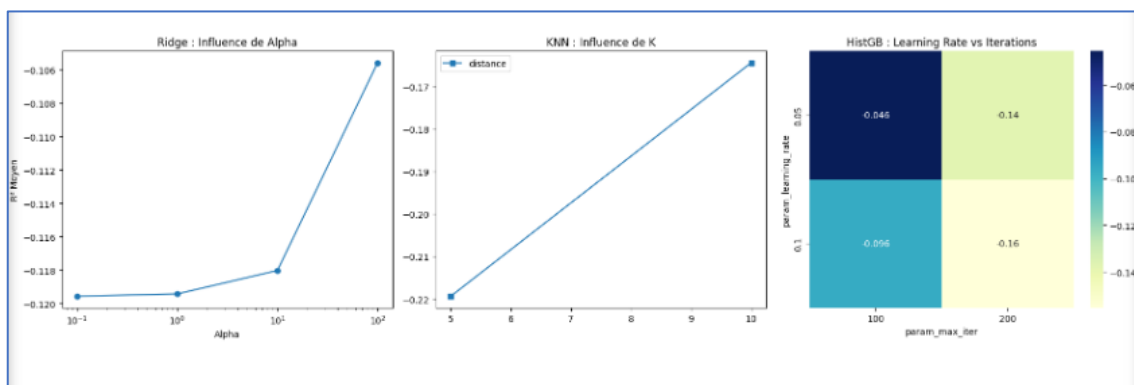
- L'influence de l'hyperparamètre Alpha est visible : le R^2 reste faible autour de 0,54 pour les petits Alpha et augmente légèrement pour Alpha = 100.
- Performance globale limitée ($R^2 = 0,546$, RMSE ≈ 265 , MAE ≈ 234), ce qui indique que le modèle linéaire régularisé ne capture pas bien les relations complexes et non linéaires du procédé.

KNN :

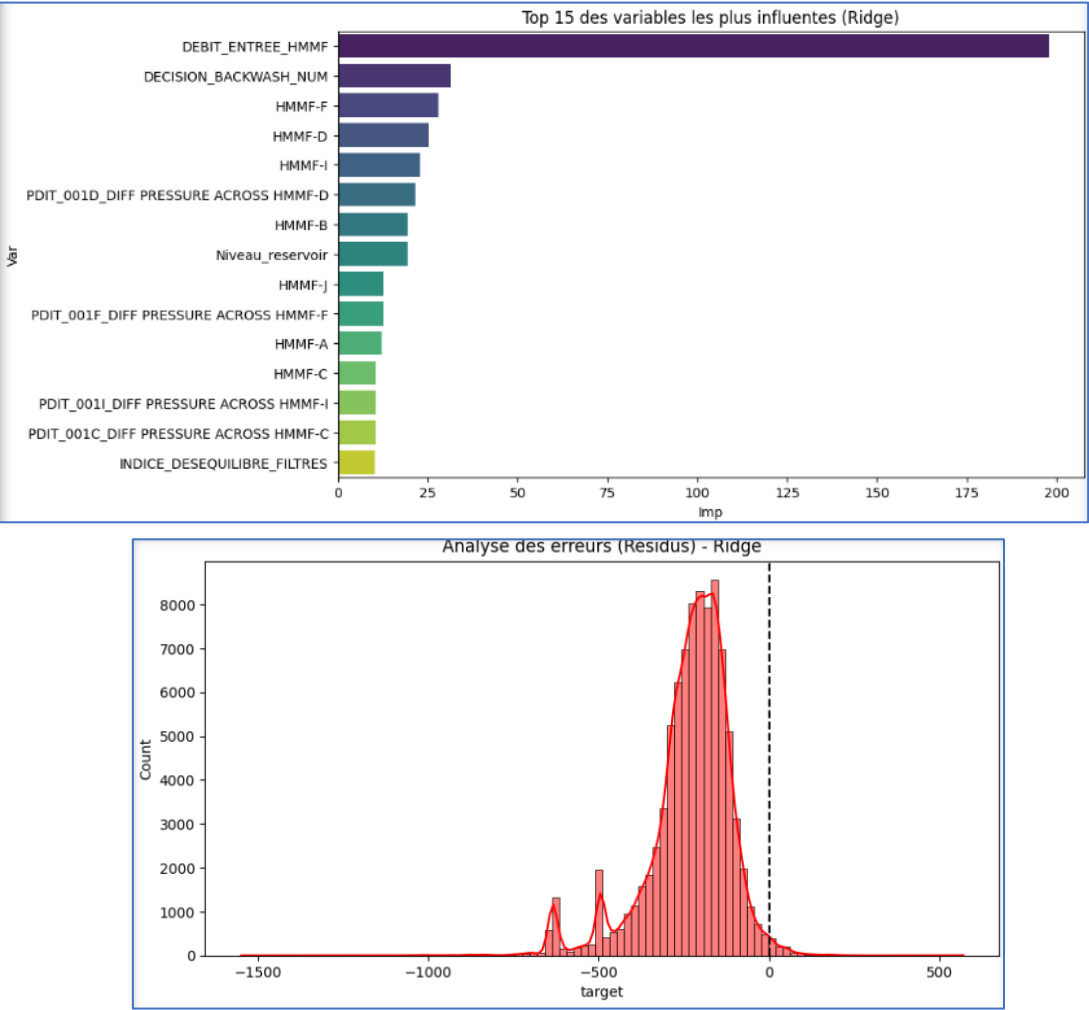
- Le graphique montre l'impact du nombre de voisins K : plus K augmente, plus la performance décroît légèrement.
- Très faible R^2 (0,186), RMSE élevé (355), MAE modéré (162).
- KNN échoue à prédire correctement les tendances sur ce dataset possiblement à cause de la complexité temporelle et de la dimensionnalité élevée.

HistGradientBoosting (HistGB) :

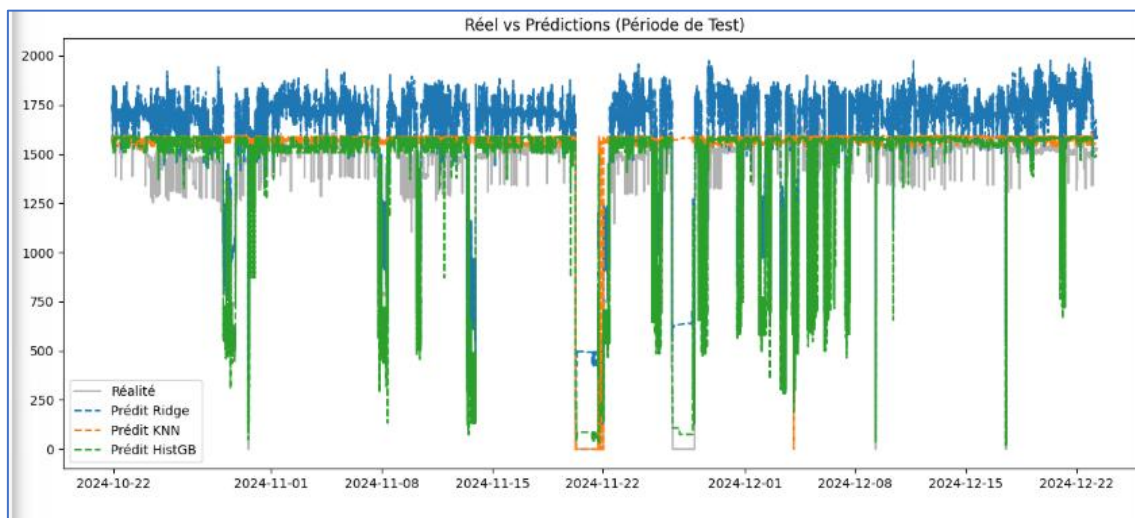
- Heatmap montre l'effet combiné du learning_rate et de max_iter sur le R^2 . Le meilleur compromis est avec learning_rate = 0.1 et max_iter = 200 ($R^2 \approx 0.917$).
- Performance nettement supérieure : $R^2 = 0,917$, RMSE ≈ 113 , MAE ≈ 72 .
- Ce modèle capture très bien les interactions complexes et non linéaires, ainsi que les dynamiques du procédé industriel.



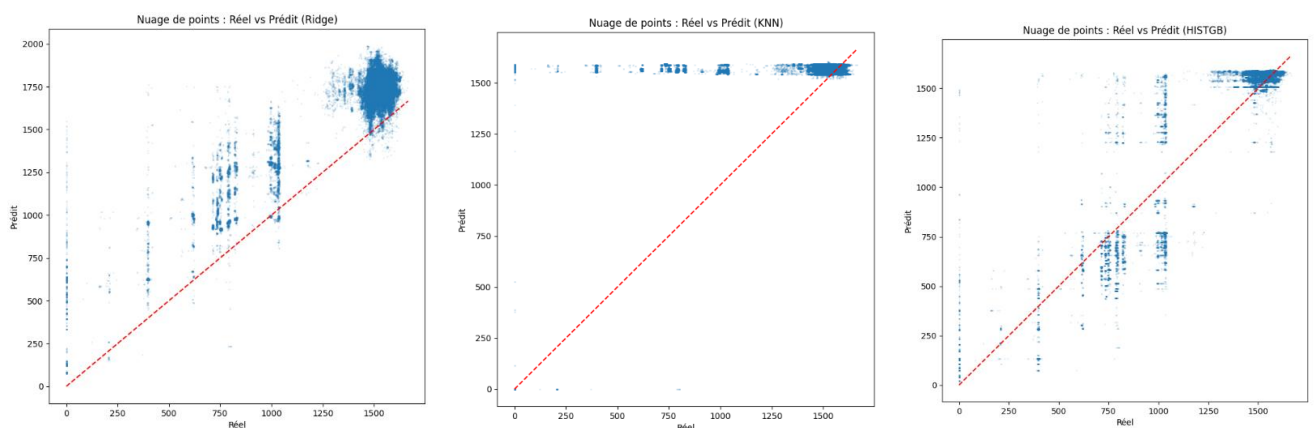
Analyse des variables et résidus : Les coefficients de Ridge révèlent les variables les plus influentes, tandis que la distribution des résidus permet de détecter les erreurs et biais.



Visualisation des résultats : Les prédictions sont comparées aux valeurs réelles via des graphiques temporels et des nuages de points, confirmant la fidélité et la robustesse des modèles.



Cette approche complète, combinant nettoyage, normalisation, optimisation par **GridSearchCV** et analyses visuelles, permet de prédire avec précision la production et de poser les bases pour une stratégie de backwash intelligente et basée sur les données.

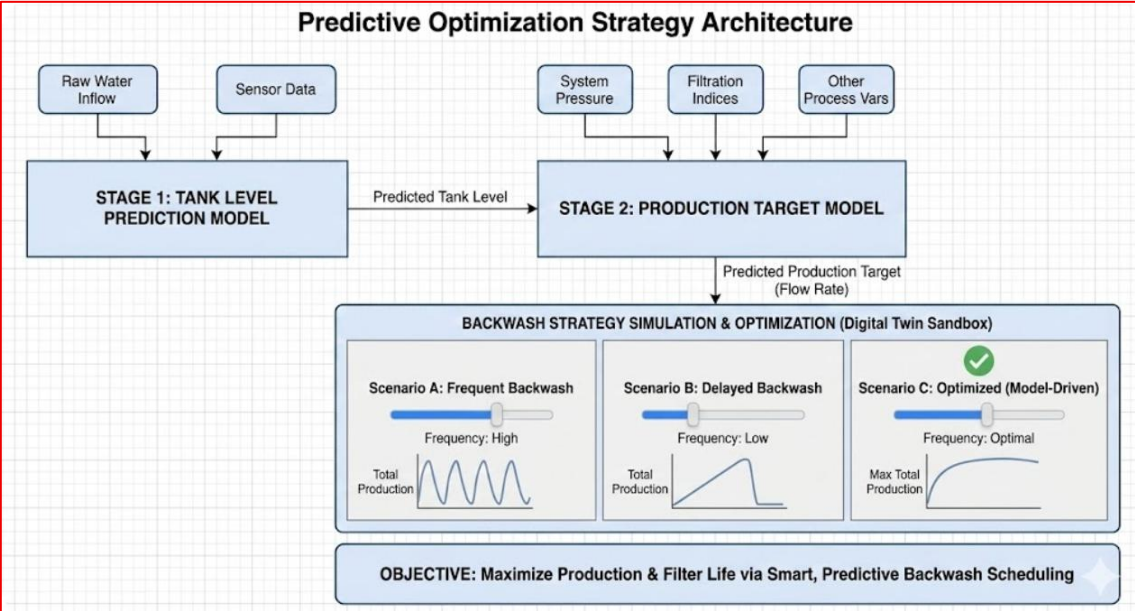


Ainsi, le modèle choisi est **Ridge**.

On a d'abord construit un **premier modèle en début qui permet de prédire le niveau du Tank**, choisi pour sa capacité à capter les tendances principales du système. Ensuite, on a créé un **modèle prédictif secondaire Ridge** qui utilise la valeur déjà prédite par le premier modèle, ainsi que les autres colonnes et indices créés, pour estimer la **production finale (target)**.

À partir de ce modèle, il devient possible de **tester différentes stratégies de backwash**. En modifiant les paramètres et colonnes spécifiques, **on simule plusieurs combinaisons** et on évalue leur impact directement grâce aux prédictions de notre modèle sur la production finale.

L'objectif est de **déterminer la combinaison optimale** : un backwash programmé de manière intelligente, prenant en compte le niveau du réservoir et l'arrivée d'eau, permettant de maximiser la production tout en préservant les filtres. Cette approche clôt le projet en fournissant une **stratégie opérationnelle optimale et basée sur la prédiction**.



Chapitre 5 : Pipeline de prédiction Machine learning en cascade

Le pipeline implémente une prédiction en deux étapes (cascade) : d'abord estimation du **niveau du réservoir (tank)**, puis prédiction de la **production finale** en s'appuyant sur la valeur de tank prédite et sur d'autres variables opérationnelles.

Organisation générale

- Deux listes de colonnes sont définies : `tank_list` (capteurs utilisés pour estimer le niveau du réservoir : pressions, débits et ΔP HMMF) et `features_list` (entrées du modèle final : Niveau_reservoir, paramètres de backwash et indices calculés).
- Les modèles, scalers et l'imputer sont sérialisés sur disque et chargés via `joblib.load` au moment de la prédiction.

Fonction `extract_features_from_df`

- Entrées : un DataFrame complet (`df`), un indice de ligne (`index`), `tank_list` et `features_list`.
- Rôle : extraire la ligne choisie et construire un dictionnaire `{col: [valeur]}` contenant uniquement les colonnes présentes dans le DataFrame. Ce format (valeurs en listes) est adapté pour construire un `pd.DataFrame` d'une seule observation utilisable par les modèles.

Fonction `predict_tank_and_prod`

- Entrées : `data_input` (dictionnaire de valeurs), `tank_list`, `features_list`, et le nom de la colonne cible du tank dans `features_list` (par défaut 'Niveau_reservoir').
- Étapes :
 1. **Chargement** des objets : modèle tank (`best_model_tuned.pkl`), scaler tank, modèle production (`model_Ridge_final.pkl`), scaler production et imputer.
 2. **Préparation pour le modèle tank** : construction d'un DataFrame `X_tank` à partir des colonnes de `tank_list`, transformation par `tank_scaler` puis prédiction `tank_pred = tank_model.predict(...)`.
 3. **Injection** : la prédiction `tank_pred` est insérée dans `data_input` sous la clé `tank_col_in_features` (ex. 'Niveau_reservoir').
 4. **Préparation pour le modèle final** : création de `X_prod` réindexé selon `features_list` (ordre fixe attendu par le pipeline d'entraînement).
 5. **Imputation et normalisation** : application de l'imputer, puis du `prod_scaler`.
 6. **Prédiction finale** : `final_pred = prod_model.predict(X_scaled)`.
- Sortie : tuple (`tank_pred`, `final_pred`) correspondant respectivement au **niveau du réservoir estimé** et à la **production finale prédite**.

[2]:

df.head()

[2]:

	Niveau_reservoir	DECISION_BACKWASH_NUM	DP_moyen_filtres	type_backwash	duree_backwash	intervalle_min	freq_backwash	INDICE_ENCRASSEMENT_HM
time								
2024-02-13 11:38:00	82.893753	3	0.695475	3	271.0	NaN	734.0	0.695475
2024-02-13 11:39:00	82.375000	3	0.694425	3	271.0	1.0	734.0	0.694425
2024-02-13 11:40:00	81.356247	3	0.693700	3	271.0	1.0	734.0	0.693700
2024-02-13 11:41:00	81.487503	3	0.692450	3	271.0	1.0	734.0	0.692450
2024-02-13 11:42:00	81.237503	3	0.693088	3	271.0	1.0	734.0	0.693088

voici ici quelques résultats de calcul suivant ce pipeline :

	Niveau_reservoir	target
time		
2024-02-13 11:38:00	82.893753	1437.231247
2024-02-13 11:39:00	82.375000	1441.015625
2024-02-13 11:40:00	81.356247	1440.490616
2024-02-13 11:41:00	81.487503	1440.512512
2024-02-13 11:42:00	81.237503	1440.774994

Chapitre 6 : le Pipeline avec Deep Learning

Tout ce pipeline que nous avons développé avec des modèles classiques (Ridge) sera reproduit de manière analogue en utilisant des **modèles de Deep Learning**. Pour cela, nous allons construire un pipeline similaire en cascade, où le niveau du réservoir sera d'abord prédit par un modèle DL, puis injecté dans un second modèle DL pour estimer la production finale.

Afin d'optimiser les hyperparamètres et tester plusieurs configurations, nous appliquerons **GridSearchCV** sur les modèles DL. Cela permettra d'évaluer différentes architectures, fonctions d'activation, nombres de neurones et taux d'apprentissage, afin de sélectionner la combinaison offrant les meilleures performances sur nos données.

1. Préparation des données :

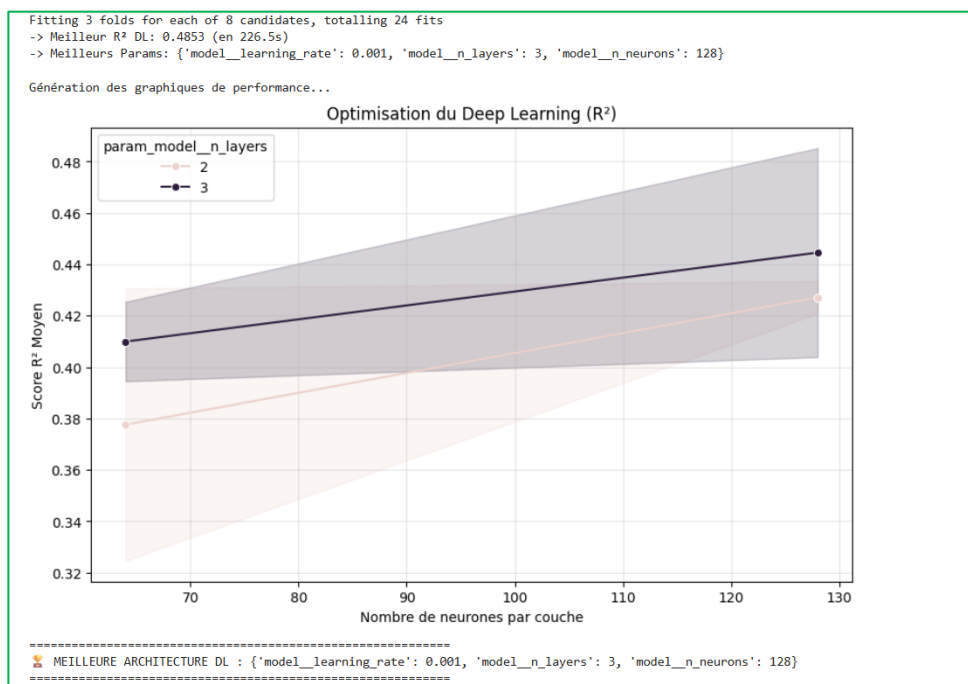
Les données sont chargées et standardisées à l'aide de StandardScaler pour faciliter la convergence du réseau de neurones. Un échantillon est extrait pour les tests de GridSearch afin d'accélérer l'optimisation.

2. Définition du modèle DL :

Nous avons créé une fonction `create_dl_model` qui construit un réseau Sequential avec un nombre variable de couches et de neurones, des couches de Dropout pour régulariser et une couche de sortie pour la prédiction continue. La compilation se fait avec l'optimiseur Adam et la fonction de perte MSE.

3. Optimisation des hyperparamètres avec GridSearchCV :

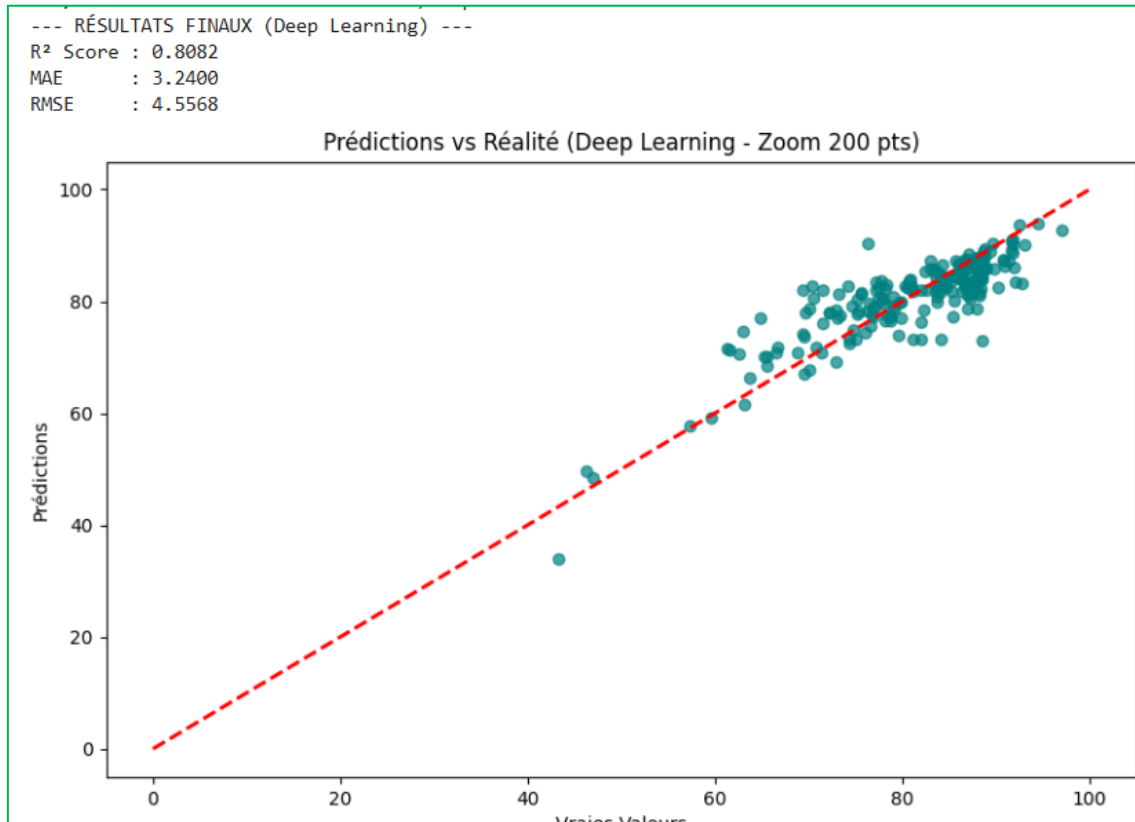
En utilisant `CompatibleKerasRegressor` (une version corrigée du wrapper `KerasRegressor`), nous avons testé différentes combinaisons de couches, neurones et



taux d'apprentissage pour identifier la meilleure architecture sur un échantillon de 15 000 lignes.

4. Visualisation des performances :

Les résultats du GridSearch sont visualisés pour observer l'influence du nombre de couches et de neurones sur le R^2 moyen.



5. Entraînement final :

Le meilleur modèle identifié est entraîné sur l'ensemble du dataset avec un nombre plus important d'époques et un batch size adapté.

6. Évaluation et sauvegarde :

Les performances sont évaluées sur un jeu de test avec R^2 , MAE et RMSE, et un graphique comparant les prédictions aux valeurs réelles est généré. Enfin, le modèle et le scaler sont sauvegardés pour une utilisation future.

Un exemple d'usage :

```
=====
VÉRIFICATION : MODÈLE CHARGÉ VS RÉALITÉ
=====
VALEUR RÉELLE (Ligne 1) : 82.8938
PRÉDICTION DL           : 84.0973
-----
ERREUR ABSOLUE          : 1.2035
PRÉCISION                : 98.55 %
=====
```

Après avoir établi le modèle Deep Learning pour le niveau du Tank, nous avons construit **le second modèle prédictif**, dédié à la **production finale**, en suivant la même approche, mais adapté à ce nouvel objectif.

1. Préparation des données :

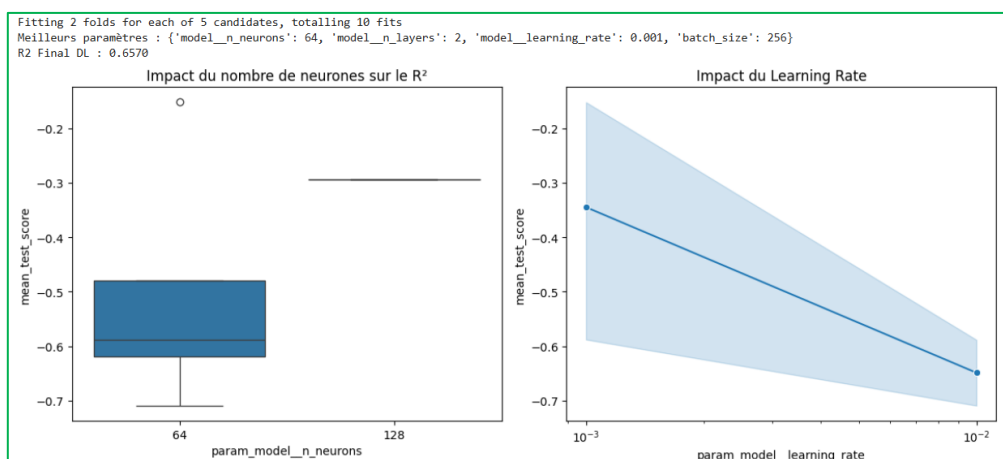
Les données sont triées par date, la colonne cible target est séparée des features, et le dataset est divisé en train/test sans shuffle pour conserver l'ordre temporel. Les valeurs manquantes sont imputées par la moyenne et les features sont standardisées.

2. Construction du modèle DL :

Une fonction create_model définit un réseau avec un nombre variable de couches et de neurones, des Dropout pour régulariser et une couche de sortie unique pour la régression. L'optimiseur Adam et la perte MSE sont utilisés pour l'apprentissage.

3. Optimisation des hyperparamètres :

Nous utilisons RandomizedSearchCV avec un wrapper KerasRegressor pour tester différentes architectures (nombre de couches, neurones, taux d'apprentissage et batch size) sur un TimeSeriesSplit. Cela permet de choisir automatiquement la meilleure configuration tout en respectant la nature temporelle des données.



4. Évaluation et visualisation :

Le modèle optimisé est évalué sur le jeu de test avec le R^2 final. Des graphiques montrent l'impact du nombre de neurones et du learning rate sur le score R^2 , ainsi

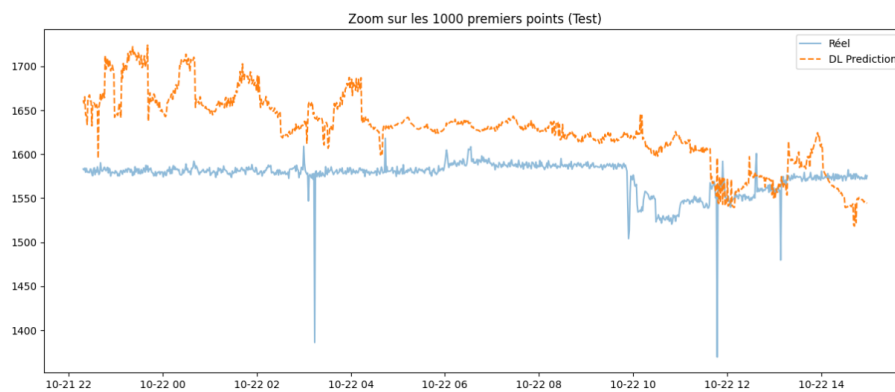
qu'une comparaison entre les valeurs réelles et les prédictions sur un zoom des 1000 premiers points.

5. Sauvegarde :

Le modèle final, ainsi que les objets scaler et imputer, sont sauvegardés pour une utilisation future dans le pipeline complet.

Le pipeline final :

Après avoir créé le premier modèle Deep Learning pour prédire le niveau du **Tank**, nous avons construit le **second modèle DL**, dédié à la **production finale**, en utilisant comme entrée les valeurs du Tank prédit ainsi que les autres features et indices calculés.



Pipeline du premier modèle (Tank) :

- Les données brutes sont extraites ligne par ligne ou par lot.
- Les features sont standardisées à l'aide du scaler précédemment entraîné.
- La prédiction du Tank est effectuée via le modèle DL sauvegardé (best_model_dl_tuned.pkl).
- Cette prédiction est injectée comme nouvelle feature dans le second modèle.

Création du second modèle (Production finale) :

- Les données incluent maintenant la colonne Niveau_reservoir issue du premier modèle.
- Les autres colonnes spécifiques à la production finale sont sélectionnées (features_list).
- Les valeurs manquantes sont imputées et toutes les features sont standardisées avec les objets imputer et scaler du modèle final.
- Le réseau de neurones est entraîné via KerasRegressor avec hyperparamètres optimisés par RandomizedSearchCV.
- Le modèle prédit la production finale pour chaque ligne du dataset, y compris les nouvelles combinaisons simulées de paramètres (type_backwash, durée, intervalle, etc.).

Pipeline complet en cascade :

- La prédiction du Tank alimente directement le second modèle.
- Le modèle final peut évaluer toutes les combinaisons possibles de paramètres opérationnels.
- Les prédictions sont ajoutées au dataset pour analyse et sauvegardées pour exploitation future.

2. Vérifications et évaluation :

- Sur le nouveau dataset, on compare les valeurs réelles et les prédictions pour tester la cohérence.
- Des métriques comme R^2 , MAE et RMSE permettent d'évaluer la performance globale du pipeline.
- Un test rapide sur la première ligne assure que le flux de données et les prédictions restent corrects.

Quelques exemples d'évaluation :

4084	3	60.00	10.0	60.0	1575.4785
3769	3	60.00	10.0	60.0	1573.7946
3089	3	60.00	10.0	60.0	1573.0948
3929	3	60.00	10.0	60.0	1571.2300
3129	3	60.00	10.0	60.0	1568.8517

Chapitre 7 : Optimisation entre Machine learning et Deep learning

Recherche de la combinaison optimale

Après avoir construit notre **pipeline complet** — avec un modèle **ML** et un modèle **DL**, tous deux conçus pour prédire le **niveau du Tank** et la **production finale** — nous entrons dans la phase finale : **optimiser la production**.

1. Principe général

- Certains paramètres opérationnels ne sont pas prédits par les modèles (ex.valeur de flux , ph , conductivité ...), et d'autres peuvent être fixés à des **valeurs moyennes** ou standards.
- Les indices créés pendant l'analyse (ex. INDICE_ENCRASSEMENT_HMMF, INDICE_IMPACT_PRODUCTION, INDICE_DESEQUILIBRE_FILTRES) deviennent les **leviers d'optimisation**.

2. Approche utilisée

- Nous générons toutes les **combinaisons possibles** des indices modifiables dans un intervalle réaliste.
- Chaque combinaison est passée dans notre **pipeline complet** pour prédire :
 1. Le niveau du Tank (via le premier modèle).
 2. La production finale correspondante (via le second modèle).

3. Objectif d'optimisation

- Identifier la combinaison de paramètres et d'indices qui **maximise la production finale**.
- Comparer les prédictions pour chaque scénario afin de **trouver les conditions idéales**.

4. Avantages de cette approche

- Permet une **simulation virtuelle** de différentes stratégies opérationnelles sans toucher physiquement aux installations.
- Donne une **base quantitative** pour programmer le backwash et ajuster les paramètres afin de **maximiser la production tout en préservant les filtres**.
- La combinaison optimale peut ensuite servir de **référence opérationnelle** pour la prise de décision en production.

la fonction generate_all_combinations

Cette fonction sert à **générer toutes les combinaisons possibles de paramètres** pour tester différentes configurations dans notre pipeline (ML + DL) afin d'optimiser la production finale.

1. Objectif principal

- Créer un dataset synthétique regroupant toutes les **variations possibles** des variables continues et catégorielles.
- Calculer automatiquement les indices dérivés (INDICE_IMPACT_PRODUCTION) pour chaque combinaison.
- Servir de base pour la **simulation et l'optimisation** dans la phase "Endgame".

2. Étapes clés de la génération

◆ Tableau récapitulatif des fonctionnalités par type de variable				
Catégorie	Colonnes / Indices	Type	Plage / Valeurs	Particularité
Niveau du tank & flux	Niveau_reservoir, hmmf_inlet_flow, ro_permeate_flow, hmmf_diff_pressure	float	min / max	Valeurs continues générées uniformément
Backwash	DECISION_BACKWASH_NUM	int	[0,1,2,3]	Catégorielle, toutes les combinaisons générées
	duree_backwash, intervalle_min, freq_backwash	float	min / max	Continues, répétées pour chaque combinaison cat.
Indices calculés	INDICE_ENCRASSEMENT_HMMF, DEBIT_ENTREE_HMMF, INDICE_IMPACT_PRODUCTION, INDICE_DESEQUILIBRE_FILTRES, DP_moyen_filtres	float	min / max	INDICE_IMPACT_PRODUCTION calculé automatiquement
Filtres HMMF	HMMF-A ... HMMF-J	int	0 / 1	Variables binaires, toutes les combinaisons générées
Pression filtres	PDIT_001A_DIFF PRESSURE ...	float	min / max	Continues, valeurs uniformes

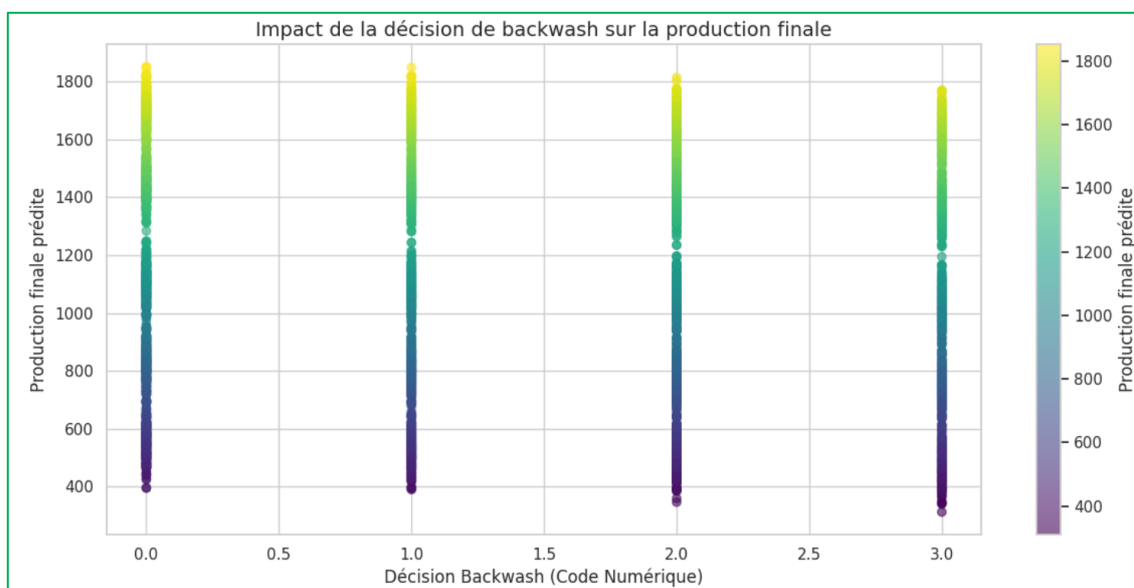
- **Séparation des colonnes :**
 - Catégorielles (allowed) → toutes les combinaisons possibles avec itertools.product.
 - Continues (min/max) → valeurs espacées uniformément avec np.linspace.
- **Répétition des valeurs continues** pour correspondre au nombre de combinaisons catégorielles.
- **Calcul automatique des indices** calculés, par exemple INDICE_IMPACT_PRODUCTION.
- **Réorganisation des colonnes** selon features_list si fourni.
- **Validation** des valeurs selon les règles (min, max, allowed).

3. Sortie

- Un DataFrame `all_combos` contenant toutes les combinaisons possibles, prêt à être injecté dans les modèles pour prédire le niveau du Tank et la production finale.
- Sauvegardé dans `all_combinations_pipeline.csv`.

Cette fonction crée un **jeu de données exhaustif** combinant toutes les variations possibles des paramètres, avec calcul automatique des indices dérivés. C'est la **base pour l'étape finale**, où l'on teste chaque combinaison pour maximiser la production finale via notre pipeline ML + DL.

Résultats : Machine learning



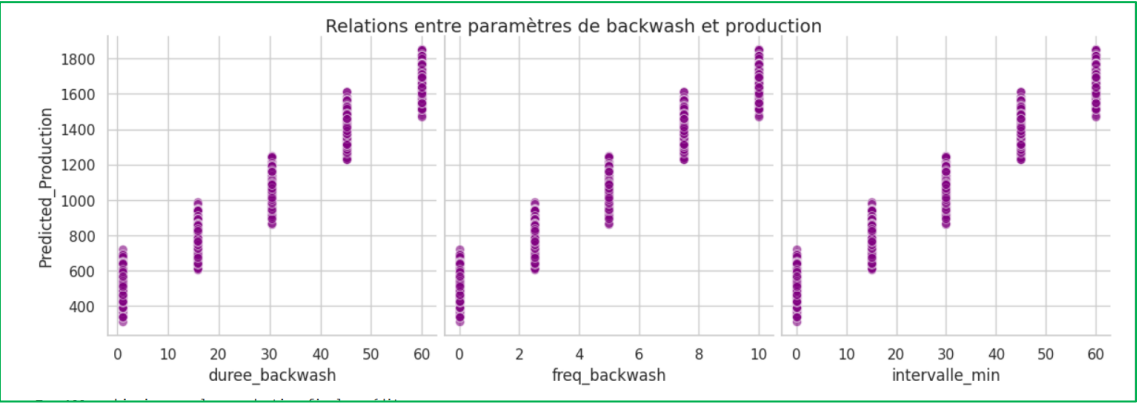
Le pipeline ML est conçu pour traiter **un très grand nombre de combinaisons de paramètres**, générées selon les règles définies pour chaque feature (valeurs continues, catégorielles ou indices calculés). Pour chaque ligne ou combinaison : le pipeline complet était appliqué .

En répétant ce processus sur **toutes les combinaisons possibles** (ou un échantillon représentatif), on peut analyser les résultats pour :

- Identifier les configurations de paramètres qui donnent les productions les plus élevées.
- Comprendre l'impact relatif de chaque indice ou variable sur la production finale.

- Fixer certaines colonnes ou donner des valeurs moyennes pour tester différents scénarios.

Ce processus permet donc de **trouver systématiquement la meilleure combinaison de paramètres et d'indices** pour maximiser la production finale, en utilisant les modèles ML existants comme simulateurs.



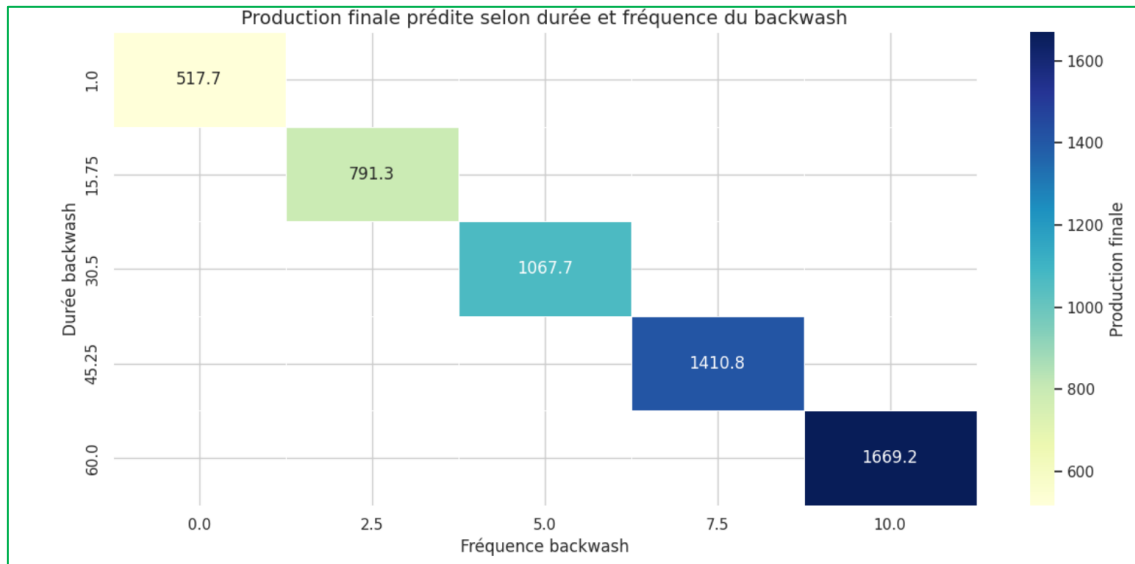
Et ici on peut vraiment trouver les meilleures combinaisons et signaux à détecter pour pouvoir vraiment avoir une meilleure production finale :

Top 100 combinaisons selon production finale prédite :

	DECISION_BACKWASH_NUM	duree_backwash	freq_backwash	intervalle_min	Predicted_Production
249	0.0	60.0	10.0	60.0	1851.926755
209	0.0	60.0	10.0	60.0	1850.406623
1269	1.0	60.0	10.0	60.0	1849.072852
244	0.0	60.0	10.0	60.0	1846.062993
509	0.0	60.0	10.0	60.0	1827.000350
469	0.0	60.0	10.0	60.0	1825.480218
629	0.0	60.0	10.0	60.0	1822.843923
89	0.0	60.0	10.0	60.0	1822.597501
1149	1.0	60.0	10.0	60.0	1821.263730
1109	1.0	60.0	10.0	60.0	1819.743598
729	0.0	60.0	10.0	60.0	1819.179345
124	0.0	60.0	10.0	60.0	1818.253871
1789	1.0	60.0	10.0	60.0	1817.845573
84	0.0	60.0	10.0	60.0	1816.733739
2549	2.0	60.0	10.0	60.0	1777.150355
504	0.0	60.0	10.0	60.0	1776.994399
1229	1.0	60.0	10.0	60.0	1776.116185
1649	1.0	60.0	10.0	60.0	1775.847831
464	0.0	60.0	10.0	60.0	1775.474267
2169	2.0	60.0	10.0	60.0	1774.267638
1524	1.0	60.0	10.0	60.0	1774.140496
204	0.0	60.0	10.0	60.0	1773.106326
624	0.0	60.0	10.0	60.0	1772.837972
2129	2.0	60.0	10.0	60.0	1772.747506
48	0.0	60.0	10.0	60.0	1771.700500

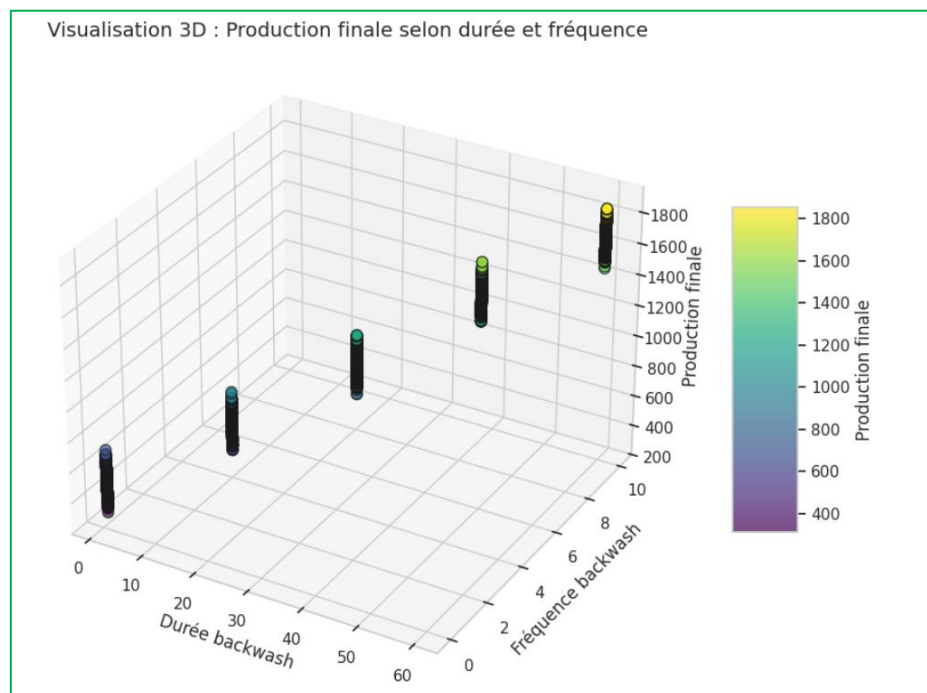
Il reste à en choisir ceux logiques et en même temps maximal : comme la décision de faire un backwash partiel avec 60 minutes aux filtres A Backwasher repartis sur des 10 fois ce jour-là .

En voici d'autres possibles :



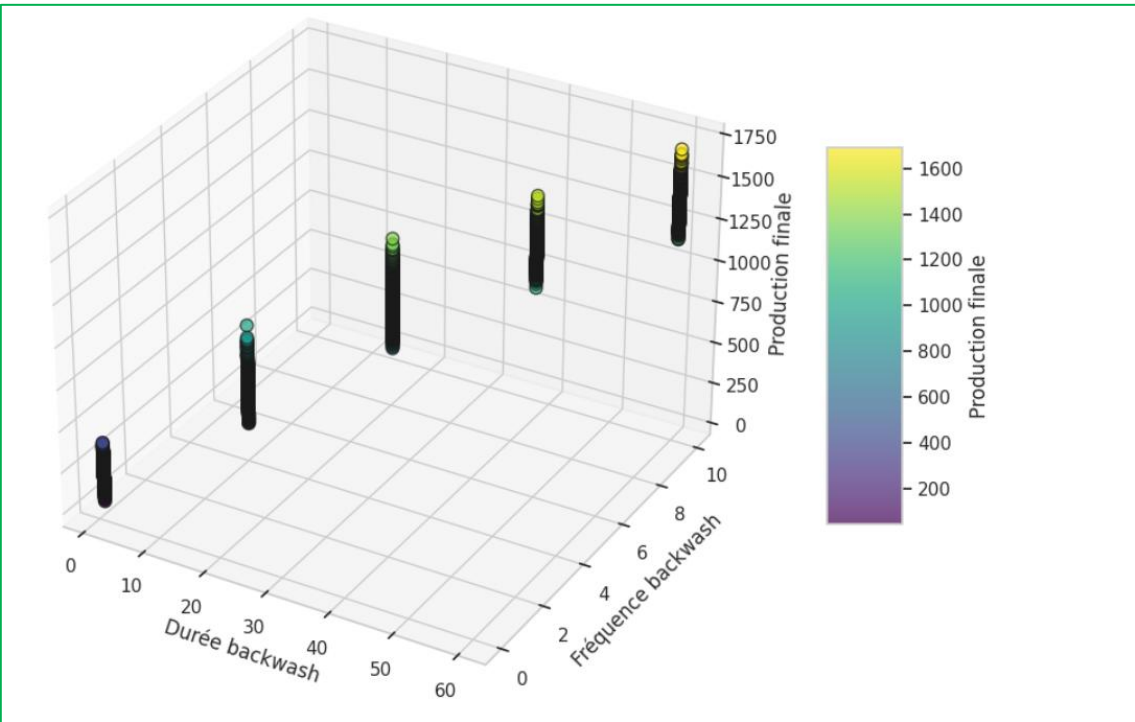
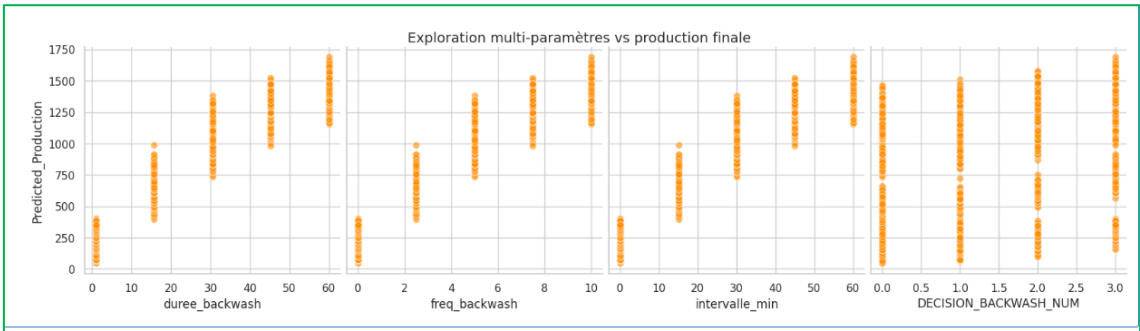
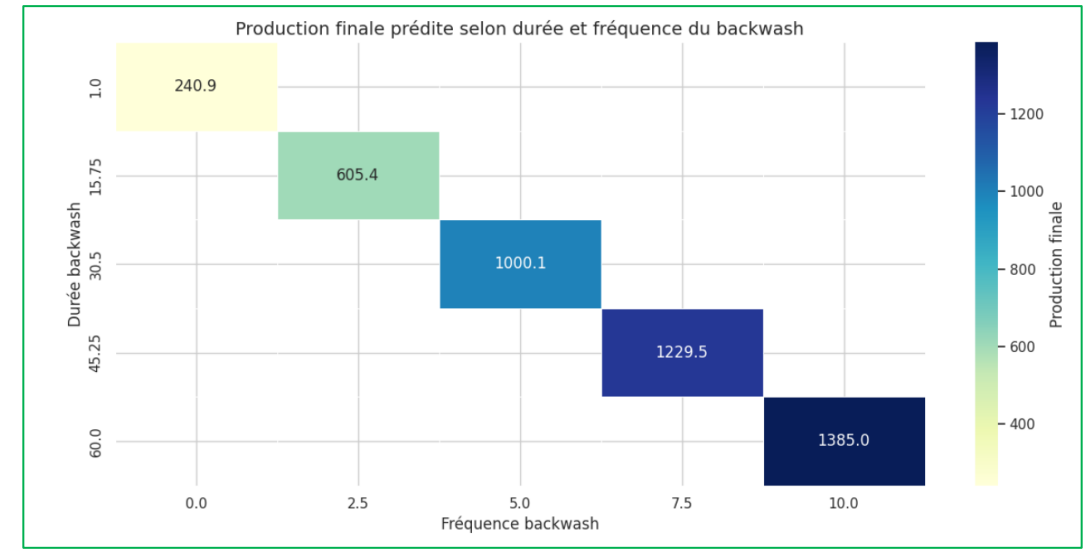
et ceci nous prouve que : le backwash partiel fait aux temps aux indices des filtres le signalent avec une durée de 60 minutes sur 6 à 10 fois sera très optimal pour la production finale : on pourra nous fixer que nous avons arrivé à atteindre presque

pourcentage ML = $1850 / (240 * 8)$ 96% de la capacité de production finale



Résultats : Deep learning

Le même chose était faite avec le deep learning de la même manière, il a donne des résultats presque similaire mais disant plus solides et réalistes :



	DECISION_BACKWASH_NUM	duree_backwash	freq_backwash	intervalle_min	Predicted_Production
4089	3	60.00	10.0	60.0	1689.4976
3289	3	60.00	10.0	60.0	1661.5034
3569	3	60.00	10.0	60.0	1655.5619
3529	3	60.00	10.0	60.0	1652.8848

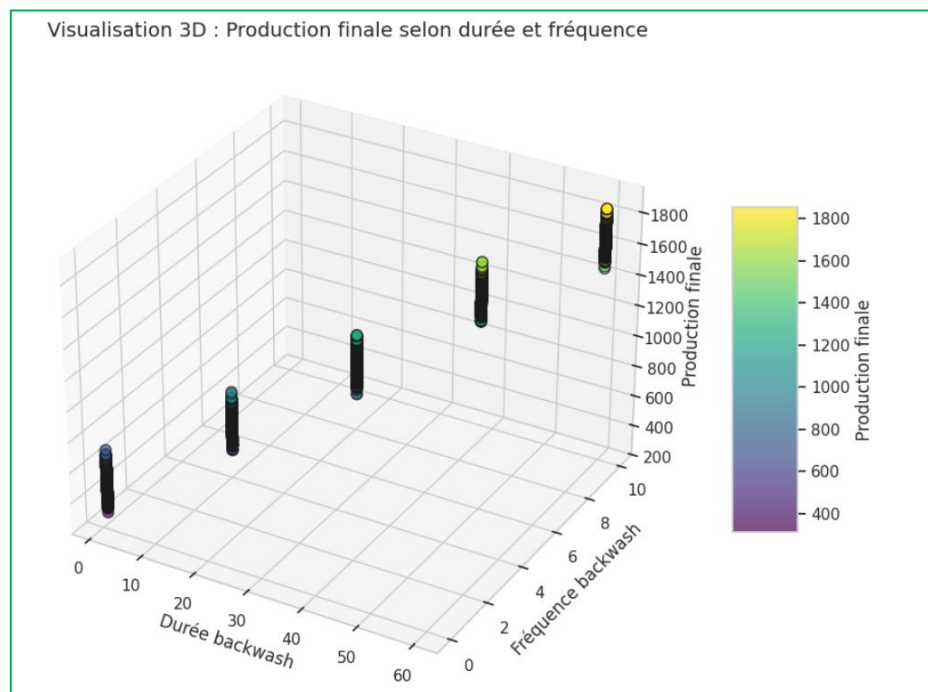
.....

3744	3	60.00	10.0	60.0	1508.6090
2544	2	60.00	10.0	60.0	1507.8607
3553	3	45.25	7.5	45.0	1507.5497
2264	2	60.00	10.0	60.0	1507.2770
4019	3	60.00	10.0	60.0	1506.5211
3709	3	60.00	10.0	60.0	1505.5419

et ceci nous prouve que : le backwash partiel fait aux temps aux les indices des filtres le signalent avec une duree de 45-60minutes sur 6 à 10 fois sera très optimal pour la production finale : on pourra nous fixer que nous avons arrivé a atteindre presque soit en ML ou DL

pourcentage DL = $1600 / (240 * 8)$

83% de la capacite de production finale



Conclusion :

Le projet de dessalement vise à optimiser la production finale d'eau traitée en contrôlant différents paramètres opérationnels des réservoirs et filtres HMMF, notamment les décisions de backwash, la durée, la fréquence et d'autres indices calculés liés à l'état des filtres. Dans ce contexte, la data science joue un rôle crucial : elle permet de transformer des données complexes et massives en informations exploitables, d'identifier les relations entre paramètres et production, et de prédire les résultats avant même d'exécuter physiquement les opérations.

Pour atteindre cet objectif, un pipeline complet a été développé : un modèle ML et DL prédit le niveau du réservoir et estime ensuite la production finale à partir de ce niveau et des autres paramètres. Le pipeline a été appliqué à des centaines de milliers de combinaisons possibles de variables continues, catégorielles et indices calculés, ce qui a permis d'explorer l'ensemble de l'espace opérationnel de manière systématique.

Les résultats obtenus ont montré quelles configurations maximisent la production finale. Les visualisations, incluant des scatter plots, pairplots, heatmaps et graphiques 3D, ont mis en évidence l'influence des paramètres clés et les interactions complexes entre eux.

En conclusion, ce projet illustre l'importance de la data science dans le secteur du dessalement : elle fournit un outil puissant pour l'optimisation opérationnelle, permettant non seulement de prédire la production mais aussi de proposer les meilleures stratégies pour maximiser l'efficacité et la performance du système.